

SynMax Python Take-Home Architecture And 5-Day Build Plan

Prepared for Athena Nexis

Date: June 9, 2026

Summary

This project will solve the SynMax Python developer take-home in two parts:

- Part One: scrape or otherwise responsibly acquire New Mexico oil and gas well data for the provided API numbers, normalize the fields, and load the results into a SQLite table named `api_well_data`.
- Part Two: expose the stored data through a small Python API with a single-well lookup endpoint and a polygon search endpoint.

The implementation should be built step by step. This document is the durable roadmap for the project, not a signal that the whole project should be implemented at once.

Assignment Requirements

The provided CSV, `apis_pythondev_test.csv`, contains 480 oil and gas well API numbers. The assignment asks us to collect well data for each API and load one SQLite table named `api_well_data` with these exact columns:

```
Operator
Status
Well Type
Work Type
Directional Status
Multi-Lateral
Mineral Owner
Surface Owner
Surface Location
GL Elevation
KB Elevation
DF Elevation
Single/Multiple Completion
Potash Waiver
Spud Date
Last Inspection
TVD
API
Latitude
Longitude
CRS
```

Part Two requires:

- A `well` endpoint that returns all available database fields for a single API number.
- A geospatial polygon search endpoint that accepts latitude/longitude pairs and returns the API numbers located inside that polygon.

The final public Git repository should include:

- Scraping and database loading code.
- API code.
- Local setup instructions.
- The populated `sqlite.db` file.
- A CSV containing API numbers returned by the polygon endpoint for the assignment polygon:

```
[ (32.81, -104.19), (32.66, -104.32), (32.54, -104.24), (32.50, -104.03), (32.73, -104.01), (32.79, -103.91), (32.84, -104.05), (32.81, -104.19) ]
```

Current Source Finding

The original assignment points to the NM OCD `WellDetails.aspx` page. In current testing, direct detail pages may return a Cloudflare Turnstile human-verification page instead of well data. We should not attempt to bypass human-verification controls.

The public expanded-results export endpoint is reachable and contains all 480 API numbers from the provided CSV. That makes it the best first data source for the implementation because it reduces request volume and avoids scraping hundreds of protected detail pages.

Recommended source strategy:

- Prefer the expanded-results export as the primary source.
- Keep source acquisition behind a replaceable adapter so a detail-page scraper, official EMNRD API source, or fixture source can be swapped in later.
- Fill detail-only fields from an enrichment source if it is available and acceptable.
- If a required field is blocked by the current website state, store `NULL` and document the reason clearly in the `README`.

Recommended Architecture

Use a small layered Python package:

```

apis_pythondev_test.csv
|
v
ingestion CLI
|
v
source adapter
- expanded export source
- optional detail-page source
- optional official API/fixture source
|
v
parser and normalizer
|
v
SQLite api_well_data table
|
v
FastAPI service
- GET /well/{api_number}
- GET /wells/polygon

```

Responsibilities:

- `ingest`: read the CSV, fetch/cache source data, filter to the 480 APIs, normalize records, and load SQLite.
- `storage`: own SQLite schema, exact assignment column names, idempotent upserts, and query helpers.
- `api`: expose FastAPI routes and HTTP validation.
- `geo`: parse polygon inputs and run point-in-polygon checks.
- `cli`: provide commands such as `load-db`, `serve`, and `polygon-csv`.

This design is intentionally simple. It keeps the external website isolated from the API and database logic, which makes the project easier to test and easier to explain.

SQLite Design

Create a single required table:

```

CREATE TABLE IF NOT EXISTS api_well_data (
    "Operator" TEXT,
    "Status" TEXT,
    "Well Type" TEXT,
    "Work Type" TEXT,
    "Directional Status" TEXT,
    "Multi-Lateral" TEXT,
    "Mineral Owner" TEXT,
    "Surface Owner" TEXT,
    "Surface Location" TEXT,
    "GL Elevation" INTEGER,
    "KB Elevation" INTEGER,
    "DF Elevation" INTEGER,
    "Single/Multiple Completion" TEXT,
    "Potash Waiver" TEXT,
    "Spud Date" TEXT,
    "Last Inspection" TEXT,
    "TVD" INTEGER,
    "API" TEXT PRIMARY KEY NOT NULL,
    "Latitude" REAL,
    "Longitude" REAL,
    "CRS" TEXT
);

```

Add an index on coordinates for polygon search:

```
CREATE INDEX IF NOT EXISTS idx_api_well_data_lat_lon
ON api_well_data ("Latitude", "Longitude");
```

Use exact SQLite column names from the assignment. In Python, map them to readable snake_case names at the model or repository boundary.

Field Mapping

The expanded export provides most required fields directly or through clear mapping:

Current Operator	-> Operator
Status	-> Status
Type	-> Well Type
Work Type	-> Work Type
Mineral Owner	-> Mineral Owner
Surface Owner	-> Surface Owner
Projection	-> CRS
True Vertical Depth	-> TVD
Elevation	-> GL Elevation
Kelly Bushing	-> KB Elevation
Drilling Floor	-> DF Elevation
Spud Date	-> Spud Date
Last Inspection	-> Last Inspection
Latitude	-> Latitude
Longitude	-> Longitude
API	-> API

Build Surface Location from the available location fields, such as unit letter, section, township, range, OCD unit letter, and footages.

Fields likely requiring detail-page or official API enrichment:

- Directional Status
- Multi-Lateral
- Single/Multiple Completion
- Potash Waiver

If those fields cannot be responsibly retrieved from the current public site, leave them as NULL and document that the detail source was gated at build time.

API Design

Use FastAPI.

Endpoints:

```
GET /health
GET /well/{api_number}
GET /wells/polygon?points=32.81,-104.19;32.66,-104.32;...
```

Expected behavior:

- GET /well/{api_number} returns all fields for one well.
- Unknown API numbers return HTTP 404.
- Invalid API number formats return HTTP 422 or 400.
- Polygon search accepts (latitude, longitude) pairs, validates at least three distinct points, closes the polygon if needed, filters by SQL bounding box first, and then uses exact point-in-polygon logic.
- Polygon output should be deterministic, sorted by API number, and reused by the CSV generator.

Tools

Planned tools:

```
Python 3.11+  
FastAPI  
Uvicorn  
Pydantic  
httpx  
lxml  
Shapely  
pytest  
ruff  
sqlite3  
argparse or Typer  
Git/GitHub
```

Use the Python standard library where it keeps the project simpler. Avoid adding infrastructure that does not help the take-home.

Architecture Advantages

- Small enough for a take-home while still showing mature design.
- Source adapter isolates the risky website dependency.
- Export-first ingestion is faster and places less load on the source site.
- SQLite deliverable is exactly what the assignment asks for.
- FastAPI gives clean local API docs and typed route validation.
- Shapely avoids hand-rolled geospatial edge cases.
- The same polygon logic can serve both the API endpoint and required CSV export.

Architecture Disadvantages

- The export file is large and slow to generate.
- Some fields may still require a detail-page or official API enrichment source.
- SQLite is excellent for the assignment but not ideal for high-concurrency production traffic.

- A GET polygon endpoint can become awkward for very large polygons; a POST endpoint would be better in production, but the assignment asks for GET.
- If the upstream site changes again, the parser may need adjustment, though the rest of the app should remain stable.

Five-Day Build Schedule

Day 1: Project Foundation And Data Source Proof

- Set up the repo structure, README, `pyproject.toml`, and basic package folders.
- Confirm the CSV has 480 unique API numbers.
- Confirm all 480 APIs exist in the expanded export.
- Build a parser proof-of-concept against a cached export sample.
- Document the Turnstile/detail-page risk.

Day 2: Part One Ingestion

- Implement CSV loading and API-number validation.
- Implement export download/cache behavior.
- Implement streaming HTML table parsing.
- Map source fields into the required database schema.
- Create SQLite table and coordinate index.
- Insert or upsert all 480 records.
- Add a loader summary: input count, loaded count, missing coordinate count, and null critical fields.

Day 3: Part Two API

- Implement database query helpers.
- Build GET `/health`.
- Build GET `/well/{api_number}`.
- Build polygon parsing and validation.
- Build GET `/wells/polygon`.
- Reuse polygon logic to generate the required CSV.

Day 4: Tests And Polish

- Add tests for CSV validation.
- Add tests for export parser mapping.
- Add tests for SQLite schema and row count.
- Add endpoint tests for success, invalid input, and not found cases.

- Add polygon tests for inside/outside/boundary behavior.
- Run `ruff` and clean up naming, typing, and docstrings.

Day 5: Final Verification And Submission Prep

- Rebuild the database from a clean checkout.
- Run the test suite.
- Start the local API and manually call both required endpoints.
- Regenerate the polygon CSV.
- Review README instructions from a fresh-user perspective.
- Commit, push to a public GitHub repo, and prepare the submission email.

Step-By-Step Implementation Order

Implement the project one step at a time:

1. Minimal project setup and durable plan.
2. CSV validation utilities.
3. Export fetch/cache and parser.
4. SQLite schema and loader.
5. Database verification report.
6. Single-well API endpoint.
7. Polygon search endpoint.
8. Polygon CSV generator.
9. Tests, README, and final cleanup.

Each step should be verified before moving to the next one.

Acceptance Criteria

The final project is complete when:

- `data/sqlite.db` contains table `api_well_data`.
- The table contains one row for each valid input API.
- All required assignment columns exist with exact names.
- `GET /well/{api_number}` returns all available fields for an existing API.
- Polygon search returns the correct API numbers for a provided polygon.
- The required polygon CSV exists and was generated from the same logic as the API.
- README instructions allow a reviewer to create the database and run the API locally.

- Tests pass locally.

Current Assumptions

- The project will be implemented gradually, not all at once.
- The docs plan in this file is the canonical saved plan for future conversations.
- No separate root PLAN.md is needed.
- The expanded export remains the primary planned source unless it becomes unavailable.
- Detail-only fields may be NULL if current public access prevents responsible retrieval.